Compiler Design Lab: Viva Preparation

Questions are in bold, answers follow in normal text. Section A covers general theory, then each lab program has its own section.

A. General Compiler Theory

Q1. What is a compiler?

A compiler is a program that translates a source program written in a high-level language into an equivalent target program (machine code or assembly), and reports errors found in the source.

Q2. What are the phases of a compiler?

Lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimisation and code generation. The symbol table and the error handler are used by all phases.

Q3. What is the difference between a compiler and an interpreter?

A compiler translates the whole program first and then it is executed. An interpreter translates and executes the program one statement at a time. Compiled code usually runs faster.

Q4. What is lexical analysis?

It is the first phase. It reads the source program character by character and groups the characters into meaningful units called tokens. It also removes whitespace and comments.

Q5. What is the difference between a token, a lexeme and a pattern?

A token is a category such as identifier or keyword. A lexeme is the actual character sequence in the source, for example `count`. A pattern is the rule (usually a regular expression) that describes the lexemes of a token.

Q6. What is syntax analysis?

The second phase. The parser takes the tokens and checks whether they follow the grammar of the language, producing a parse tree or reporting syntax errors.

Q7. What is a context free grammar?

A grammar with four parts: a set of terminals, a set of non-terminals, a set of productions of the form A -> alpha, and a start symbol. Every production has a single non-terminal on the left side.

Q8. What is the difference between top-down and bottom-up parsing?

Top-down parsing starts from the start symbol and tries to derive the input (recursive descent, LL parsers). Bottom-up parsing starts from the input and reduces it back to the start symbol (shift-reduce, LR parsers, YACC).

Q9. What is ambiguity in a grammar?

A grammar is ambiguous if some string has more than one parse tree. For example `E -> E + E | E * E | id` is ambiguous. It is resolved by rewriting the grammar or by giving precedence and associativity rules.

Q10. What is left recursion and why is it a problem?

A grammar is left recursive if it has a production `A -> A alpha`. A top-down parser would call itself forever without consuming any input. It is removed by rewriting as `A -> beta A'` and `A' -> alpha A' | epsilon`.

Q11. What is the symbol table?

A data structure that stores information about identifiers such as name, type, scope and memory location. It is used by most phases of the compiler.

B. Program 1: Lexical Analyzer in C

Q12. What does this program do?

It reads `input.c` one character at a time and classifies each lexeme as a keyword, identifier, number, operator, symbol or unknown, then prints it.

Q13. How do you tell a keyword from an identifier?

First the whole word is collected into a buffer. Then it is compared with every string in the keyword array using `strcmp`. If it matches, it is a keyword, otherwise it is an identifier.

Q14. Why do you use `ungetc`?

To read a word or number, the program has to read one character beyond its end to know it has finished. That extra character belongs to the next token, so `ungetc` puts it back into the input stream.

Q15. Why is `ch` declared as `int` and not `char`?

`fgetc` returns an `int` so that EOF (which is -1) can be told apart from a valid character. Storing it in a `char` could make EOF comparison fail.

Q16. Why do you check `i < 49` while building the identifier?

The buffer `str` holds 50 characters. Stopping at 49 leaves room for the terminating `'\0'` and prevents buffer overflow on very long identifiers.

Q17. What does `isalnum`, `isalpha`, `isdigit` and `isspace` do?

They are functions from `ctype.h`. They test whether a character is alphanumeric, alphabetic, a digit or whitespace.

Q18. What are the limitations of this program?

It does not handle comments, string literals, floating point numbers, or multi-character operators such as == and <=. These would be reported as separate single-character operators.

C. Program 2: Lexical Analyzer using Lex

Q19. What is Lex?

Lex is a tool that generates a lexical analyzer in C from a specification written as regular expressions with actions.

Q20. What are the three sections of a Lex program?

The definition section (between `%{` and `%}`, plus named patterns), the rules section (between the first `%%` and second `%%`), and the user subroutine section (C code such as `main` and `yywrap`).

Q21. What does `yylex()` do?

It is the scanning function generated by Lex. It matches the input against the rules and runs the action of the matching rule, continuing until the end of input.

Q22. What is `yytext`?

A character pointer that holds the lexeme matched by the current rule.

Q23. What is `yyin`?

The file pointer from which Lex reads input. By default it is `stdin`. In this program it is set to the file `input.txt`.

Q24. What is `yywrap()` and why does it return 1?

Lex calls it when it reaches the end of input. Returning 1 means there is no more input to process, so scanning stops. Returning 0 would tell Lex to continue with another input file.

Q25. What happens if two rules match the same lexeme?

Lex picks the longest match. If two rules match the same length, it picks the rule that is written first. That is why the keyword rule is placed before the identifier rule.

Q26. What does the rule `.` do?

The dot matches any single character except newline. Placed last, it catches every character that no earlier rule matched, and the program reports it as unknown.

Q27. What are the commands to run a Lex program?

`lex file.l` creates `lex.yy.c`, then `gcc lex.yy.c` compiles it, and `./a.out` runs it. On some systems `-ll` or `-lfl` is added while compiling.

D. Program 3: YACC Expression Validator

Q28. What is YACC?

YACC (Yet Another Compiler Compiler) is a tool that generates an LALR(1) bottom-up parser in C from a grammar specification.

Q29. How do Lex and YACC work together?

Lex scans the input and returns tokens. YACC calls `yylex()` whenever it needs the next token and then matches the token stream against the grammar.

Q30. What is the purpose of `y.tab.h`?

It is created by `yacc -d` and contains the numeric codes of the tokens declared with `%token`. The Lex file includes it so both files agree on token values.

Q31. What are the sections of a YACC program?

The declaration section (C code, `%token`, precedence), the grammar rules section, and the user subroutine section (`main`, `yyerror`).

Q32. What does `yyparse()` do?

It is the parser function generated by YACC. It returns 0 if the input is accepted and non-zero on failure.

Q33. What is `yyerror()`?

A function you write. YACC calls it when it finds a syntax error, so it is where the error message is printed.

Q34. Why is the grammar `expr : expr '+' expr | ...` ambiguous and how is it handled here?

Because `a+b*c` can be parsed in more than one way. The ambiguity is resolved using `%left` and `%right` declarations, which set associativity and precedence. Operators declared later have higher precedence.

Q35. What does `%prec UMINUS` do?

It gives the rule `'-' expr` the precedence of the dummy token `UMINUS`, so unary minus binds tighter than binary minus.

Q36. What kind of parser does YACC generate?

An LALR(1) shift-reduce parser. It shifts tokens onto a stack and reduces them using grammar rules.

Q37. What are shift-reduce and reduce-reduce conflicts?

A shift-reduce conflict occurs when the parser cannot decide between shifting the next token or reducing. A reduce-reduce conflict occurs when two different rules could be used to reduce. Precedence declarations remove many shift-reduce conflicts.

E. Program 4: YACC Desk Calculator

Q38. What does this program do?

It reads arithmetic expressions line by line and prints the computed result.

Q39. What are `$$`, `$1`, `$2` and `$3`?

`$$` is the value of the non-terminal on the left of the rule. `$1`, `$2`, `$3` are the values of the first, second and third symbols on the right side. In `expr : expr '+' expr`, `$1` and `$3` are the operands and `$2` is the operator.

Q40. What is `yylval`?

A global variable used to pass the value of a token from Lex to YACC. Here Lex stores the number in `yylval` using `atoi(yytext)` before returning `NUMBER`.

Q41. Why does the grammar start with `input : | input line`?

The first alternative is empty, and the second repeats `line`. Together they allow zero or more lines, so the calculator keeps accepting expressions until end of input.

Q42. How is division by zero handled?

The action checks whether `$3 == 0`. If so, it prints an error message and sets `$$ = 0`, otherwise it computes `$1 / $3`.

Q43. What is the default type of `$$` and how can it be changed?

It is `int`. To use other types you declare `%union` and attach types to tokens and non-terminals with `%token <type>` and `%type <type>`.

Q44. How do you evaluate `2+3*4` and get 14?

`*` is declared on a later `%left` line, so it has higher precedence than `+`. YACC therefore reduces `3*4` first and then adds 2.

F. Program 5: Epsilon Closure

Q45. What is an epsilon transition?

A move from one state to another in an automaton without consuming any input symbol.

Q46. What is the epsilon closure of a state?

The set of all states that can be reached from that state using only epsilon transitions, including the state itself.

Q47. Why is a state always in its own closure?

Because zero epsilon moves is also allowed, so every state can reach itself without consuming input.

Q48. What technique does your program use?

A recursive depth-first search over the epsilon transition matrix. From the current state it visits every unvisited state that has a 1 in the matrix.

Q49. Why do you reset `visited[]` for each state?

Each closure is computed independently. Without the reset, states visited while computing an earlier closure would be missing from the next one.

Q50. Why is the `visited[]` array needed?

It prevents revisiting a state, so cycles of epsilon moves (for example q0 -> q1 -> q0) do not cause infinite recursion.

Q51. Where is epsilon closure used?

In converting an epsilon-NFA to an NFA or DFA (subset construction), and in Thompson's construction based regular expression matching.

Q52. What is the time complexity?

For one state the search checks each row of the matrix, so it is O(n squared). For all n states it becomes O(n cubed).

G. Program 9: Recursive Descent Parser

Q53. What is a recursive descent parser?

A top-down parser built from a set of mutually recursive functions, one for each non-terminal of the grammar.

Q54. Which grammar does your program use?

`E -> T + E | T - E | T`, `T -> F * T | F / T | F`, and `F -> ( E ) | id`, where `id` is a single letter.

Q55. Why is the grammar written with right recursion?

A recursive descent parser cannot handle left recursion because `E()` would call itself without consuming any input. Writing `E -> T + E` consumes `T` first, so the recursion always makes progress.

Q56. What is the effect of right recursion on associativity?

It makes operators right associative, so `a-b-c` is parsed as `a-(b-c)`. For a syntax-checking program this does not matter. For evaluation, the grammar would need to be rewritten with `E'` to keep left associativity.

Q57. How are precedence levels handled?

By the grammar structure. `*` and `/` are handled in `T`, which is called from `E`, so they bind tighter than `+` and `-`.

Q58. When is a string accepted?

When `E()` returns with no error and the index `i` is at the end of the string, meaning `input[i] == '\0'`.

Q59. Why do you check `input[i] == '\0'` at the end?

Without it, a string such as `a+b)` would be accepted since `E()` would stop at `)` without reporting an error. The check makes sure the whole input was consumed.

Q60. What is backtracking and does this parser use it?

Backtracking means trying one production and going back to try another when it fails. This parser does not backtrack, because the next input character is enough to choose the production (predictive parsing).

Q61. What is the difference between a recursive descent parser and a predictive parser?

A predictive parser is a recursive descent parser that needs no backtracking. It chooses the production by looking at the next input symbol, which needs an LL(1) grammar.

Q62. How do you remove left recursion from `E -> E + T | T`?

Rewrite it as `E -> T E'` and `E' -> + T E' | epsilon`.

H. Program 12: YACC Identifier Validator

Q63. What does this program check?

Whether the input is a valid identifier, meaning it starts with a letter or underscore and continues with letters, digits or underscores.

Q64. Which part decides that the word is an identifier, Lex or YACC?

Lex. The pattern `[a-zA-Z_][a-zA-Z0-9_]*` matches an identifier and returns the token `ID`. YACC only checks that the input is one `ID` followed by a newline.

Q65. What happens when you enter `9abc`?

Lex returns the character `9` as an ordinary token, not `ID`. The grammar expects `ID`, so YACC finds a syntax error and `yyerror()` prints Invalid identifier.

Q66. What does `return 0` inside the rule action do?

It makes `yyparse()` stop right after one valid identifier has been read, so the program ends instead of waiting for more input.

Q67. Why does `yyerror` return `int` here?

In this file it is declared as `int yyerror(char *)`. YACC only requires that the function exists and takes a message string. Any return type works as long as the declaration and definition match.

I. NFA to DFA Conversion (general questions)

Q68. What is the difference between an NFA and a DFA?

A DFA has exactly one transition for each state and input symbol. An NFA may have several transitions, or none, for a symbol, and may also have epsilon transitions.

Q69. Which method is used to convert an NFA to a DFA?

Subset construction. Each DFA state represents a set of NFA states.

Q70. What are the steps of subset construction?

Start with the epsilon closure of the NFA start state as the first DFA state. For each unmarked DFA state and each input symbol, compute the set of states reachable (move), take its epsilon closure, and add it as a new DFA state if it is new. Repeat until no new states appear. A DFA state is final if it contains an NFA final state.

Q71. What is the maximum number of states in the DFA?

If the NFA has n states, the DFA can have up to 2 to the power n states, since each DFA state is a subset of NFA states.

Q72. Do NFAs and DFAs accept the same languages?

Yes. Both accept exactly the regular languages, so every NFA has an equivalent DFA.